A.P.E.X.
Adaptive Parallel Extremal Dispatch

StrmCkr*

August 20, 2026

Abstract

A.P.E.X. (Adaptive Parallel Extremal Dispatch) is a deterministic parallel sorting framework for fixed-width integer keys and associated records.

The algorithm replaces fixed-depth radix refinement with descriptor-driven bucket analysis. Each bucket maintains population count and bitwise extremal accumulators (OR and AND), from which a Variable-Bit Mask (VBM) is derived. The VBM identifies unresolved ordering structure within each bucket, and its popcount defines the Residual Variable-Bit (RVB) measure.

Sorting decisions are made using a deterministic dispatch system that evaluates monotonic structure, residual variable-bit structure, bucket population, and implementation-defined refinement thresholds. This enables early termination, adaptive radix relocation, and tuple-based collapse without probabilistic estimation or learned heuristics.

The central contribution is a descriptor-driven refinement model in which, after mandatory linear descriptor construction, subsequent refinement work is restricted to the residual variable-bit domain of each active bucket rather than scheduled uniformly over the full key width.

1 Introduction

Sorting fixed-width integer keys is a fundamental operation in systems such as databases, analytics engines, and parallel computation frameworks [1, 2]. Radix sorting is widely used due to its predictable behavior under bounded key width and has a long history of theoretical and practical engineering work [7, 8, 9].

However, classical radix methods operate on fixed bit windows regardless of whether those bits contain meaningful variation. As a result, computation is often spent on constant regions of the key space that do not contribute to ordering.

A.P.E.X. (Adaptive Parallel Extremal Dispatch) addresses this limitation by introducing a descriptor-driven refinement model. During histogram construction, each bucket accumulates bitwise OR and AND descriptors. These descriptors define a Variable-Bit Mask (VBM), identifying only those bit positions that remain relevant to ordering within the bucket.

In addition, each bucket is evaluated for monotonic structure, enabling early termination when local ordering is already resolved.

The algorithm replaces uniform radix traversal with adaptive refinement over the active ordering domain identified by bucket descriptors. This domain shrinks dynamically as bucket descriptors identify and eliminate constant bit positions from further refinement.

The remainder of this paper defines the bucket descriptor system, dispatch hierarchy, refinement mechanisms, and correctness guarantees.

*Source code: GitHub repository.

2 Contributions

The primary contributions of A.P.E.X. are:

1. A descriptor-driven sorting model based on bucket extremal descriptors ($\mathrm{OR}_b, \mathrm{AND}_b$) and the derived Variable-Bit Mask (VBM).
2. A residual variable-bit formulation in which post-descriptor refinement is derived from unresolved ordering structure rather than scheduled uniformly over fixed key width.
3. A deterministic dispatch hierarchy combining monotonic termination, adaptive MSD refinement, adaptive LSD refinement, and tuple collapse.
4. A parallel execution model that constructs descriptors during histogram generation and enables deterministic scatter execution without runtime synchronization.
5. A formal correctness framework showing that all execution paths preserve unsigned lexicographic ordering.

3 Related Work

A.P.E.X. sits at the intersection of radix sorting, parallel distribution sorting, adaptive sorting, and practical library sort implementations. The experiments in this paper directly compare only against Java `Arrays.parallelSort` and Fastutil's parallel radix implementation. Other algorithms discussed in this section are included as literature context rather than as measured baselines.

3.1 Radix and Distribution Sorting

Classical radix sorting exploits the bounded width of integer or string keys by processing fixed digit windows rather than performing comparisons between keys [1, 2]. Engineering work on radix and American-flag-style sorting emphasizes cache behavior, in-place bucket permutation, and careful implementation of counting and scattering passes [7]. Andersson and Nilsson further study efficient radix sorting and practical implementation techniques for string and integer keys [8, 9].

A.P.E.X. shares the use of histograms, bucket offsets, and deterministic scatter with radix-sort engineering. Its main distinction is that bucket descriptors are used to determine the remaining active ordering domain after partitioning. Rather than applying a uniform fixed-depth digit schedule, A.P.E.X. computes $V_b = \mathrm{OR}_b \oplus \mathrm{AND}_b$ within each bucket and dispatches only the unresolved descriptor state.

American-flag-style sorters and RADULS2-style radix implementations are therefore relevant radix-family reference points, but they are not included as measured baselines in this paper. The evaluation is restricted to readily reproducible JVM/Windows baselines available in the benchmark environment.

3.2 Parallel Prefix and Scatter Primitives

Parallel radix and distribution sorts rely on prefix sums, reductions, and deterministic layout construction. These primitives are well studied in the parallel algorithms literature [5, 4, 6]. A.P.E.X. uses the same broad execution pattern: thread-local counting, associative descriptor reduction, prefix-layout construction, and scatter. The contribution is not a new scan primitive, but the use of reduced bucket descriptors to decide which refinement operator should follow the scan/scatter stage.

3.3 Sample Sort and Parallel Comparison Sorts

Sample sort and its descendants partition data using sampled splitters, then sort or recursively distribute the resulting buckets [10]. Super Scalar Sample Sort and later in-place parallel variants such as IPS^{4}o are highly engineered comparison-based distribution sorts that use branch-conscious classification and parallel bucket processing [11, 12]. These algorithms are important points of comparison for parallel sorting generally, but they solve a different problem: they are comparison-based and target arbitrary comparable objects.

IPS^{4}o and related sample-sort implementations are therefore discussed as related work and, where a direct conversion is available, as an external key-only baseline. The primary JVM/Windows baselines remain JDK `Arrays.parallelSort` and Fastutil `LongArrays.parallelRadixSort`; IPS^{4}o measurements are reported separately because they are key-only and come from a different implementation stack.

A.P.E.X. is specialized to fixed-width integer keys and associated records. It does not estimate splitters from samples; instead, it derives exact bitwise descriptor state while scanning the input. This makes the algorithm closer to radix sorting in ordering semantics, while retaining an adaptive dispatch behavior usually associated with more data-dependent sorting schemes.

3.4 Adaptive Sorting and Monotonic Structure

Adaptive sorting algorithms exploit existing order or duplicate structure in the input so that easier instances require less work than worst-case random inputs [13, 18]. A.P.E.X. uses a narrower notion of adaptivity. It does not attempt to measure general presortedness by comparisons or runs. Instead, it tests monotonicity and all-equal structure within radix buckets and combines those terminal cases with the residual variable-bit descriptor $N_{\mathrm{RVB}}(b)$.

The monotonic and terminal telemetry in Table 6 shows how this adaptivity appears in the implementation: sorted, reverse, all-equal, and block-sorted inputs terminate through descriptor states without scheduling LSD refinement, while mixed inputs may still expose local ascending or descending bucket regions.

3.5 Library Baselines

The direct experimental baselines in this paper are Java `Arrays.parallelSort` and Fastutil 8.5.13 `LongArrays.parallelRadixSort` [15, 17]. These are practical, widely available library implementations, but they are not identical workloads in every plotted series. The `apex-records` series sorts 16-byte key/value records, while the JDK and Fastutil baselines are key-only. The comparison is therefore intended to show practical engineering behavior under common library alternatives, not a complete survey of every state-of-the-art sorting implementation.

4 Notation and Preliminaries

We consider a sequence of fixed-width integer keys with associated payloads.

Definition 1 (Input Sequence)**.** *Let*

$$R = (t_1, t_2, \ldots, t_n)$$

be a sequence of records, where each record is

$$t = (k, v),$$

with $k \in \{0, \ldots, 2^{64} - 1\}$.

Remark 2 (LSD Radix Width)**.** *The parameter* r_{lsd} *denotes the radix width used exclusively during Least-Significant-Digit (LSD) refinement.*

It is an implementation-tunable parameter and may differ from the global MSD radix width r*, allowing independent optimization of LSD cycle granularity.*

Remark 3 (Unsigned Interpretation of Keys)**.** *All keys are interpreted as unsigned 64-bit integers in the range* $\{0, \ldots, 2^{64} - 1\}$.

All bitwise operations are performed on the underlying two's-complement representation with UNSIGNED ordering semantics.

In implementations (e.g., Java), logical right shift (`>>>`*) is used for all radix extraction to ensure that the most significant bit is treated as the highest-order unsigned bit.*

Definition 4 (Bucket Indexing)**.** *For radix width* r *and shift* s*, bucket index is defined as*

$$B_s(k) = (k \gg s) \wedge (2^r - 1).$$

Definition 5 (Bucket)**.** *A bucket is defined as*

$$R_b = \{t \in R : B_s(k) = b\}.$$

Remark 6. *All subsequent refinement operates independently within each bucket.*

The bitwise descriptor operations used throughout the paper are Boolean operations over fixed-width machine-word representations, following the standard algebraic interpretation of conjunction, disjunction, and exclusive variation [14].

Remark 7 (Symbol Convention)**.** *The symbol D_b is reserved exclusively for the full bucket descriptor $(h_b, \mathrm{OR}_b, \mathrm{AND}_b, V_b, \mu_b)$. Scalar recursive refinement depth is denoted by ρ_b.*

5 Bucket Descriptor Model

A.P.E.X. constructs a compact descriptor for each bucket during histogram generation. This descriptor replaces repeated scanning of keys during refinement.

5.1 Bucket Population

For a bucket b, let:

$$h_b = |R_b|$$

denote the number of records assigned to that bucket.

5.2 Bitwise Extremal Descriptors

Each bucket maintains two bitwise accumulators:

Definition 8 (Bucket OR Descriptor)**.**

$$\mathrm{OR}_b = \bigvee_{k \in R_b} k$$

Definition 9 (Bucket AND Descriptor)**.**

$$\mathrm{AND}_b = \bigwedge_{k \in R_b} k$$

These capture whether each bit position is consistently 0, consistently 1, or mixed across the bucket.

5.3 Variable-Bit Mask (VBM)

The Variable-Bit Mask identifies all bit positions that vary within a bucket.

Definition 10 (Variable-Bit Mask)**.** *Let $M_s = 2^s - 1$ denote the mask of unresolved radix positions under shift s. Then:*

$$V_b = (\mathrm{OR}_b \oplus \mathrm{AND}_b) \wedge M_s$$

Lemma 11 (Variable-Bit Characterization)**.** *For any bit position j,*

$$V_b[j] = 1$$

if and only if at least two keys in bucket R_b differ at bit position j.

Proof. If all keys in R_b share the same value at bit position j, then

$$\mathrm{OR}_b[j] = \mathrm{AND}_b[j].$$

Consequently,

$$(\mathrm{OR}_b \oplus \mathrm{AND}_b)[j] = 0.$$

Conversely, if at least one key contains a 0 and another contains a 1 at bit position j, then

$$\mathrm{OR}_b[j] = 1$$

and

$$\mathrm{AND}_b[j] = 0.$$

Therefore,

$$(\mathrm{OR}_b \oplus \mathrm{AND}_b)[j] = 1.$$

Applying the mask M_s restricts the result to the currently active refinement domain and does not change the characterization for positions within that domain.

Thus $V_b[j] = 1$ exactly when variation exists at bit position j.

□

This characterization uses the standard Boolean-algebraic fact that a bit position varies across a set precisely when its disjunction and conjunction differ [14].

5.4 Bucket Saturation

Residual variable-bit count alone does not determine refinement efficiency.

A bucket may possess low residual structure while still containing a large population, or conversely possess high residual structure while remaining sparsely occupied.

Consequently implementation-defined dispatch policies may combine residual variable-bit count with bucket population statistics when selecting refinement operators.

Remark 12. *Bucket saturation influences refinement efficiency but does not affect correctness.*

5.5 Residual Variable-Bit Count

Definition 13 (RVB Count)**.**

$$N_{\mathrm{RVB}}(b) = \mathrm{popcount}(V_b)$$

This measures how many bit positions still influence ordering within the bucket.

5.6 Active Ordering Domain

Definition 14 (Active Domain)**.**

$$\mathcal{A}(b) = \{j \mid V_b[j] = 1\}$$

Thus:

$$|\mathcal{A}(b)| = N_{\mathrm{RVB}}(b)$$

Remark 15. *Only bits in $\mathcal{A}(b)$ can affect future ordering decisions. All constant bits are eliminated from further refinement.*

5.7 Monotonic Classification

Each bucket is also classified by ordering structure:

Definition 16 (Monotonic State)**.** *A bucket is:*

- ***Ascending*** *if keys are in nondecreasing order,*
- ***Descending*** *if keys are in nonincreasing order,*
- ***Mixed*** *otherwise.*

Remark 17. *Monotonicity is independent of bit variability and can override further refinement.*

5.8 Monotonic Elimination

Monotonic classification constitutes a terminal optimization stage.
If

$$\mu_b = \text{Ascending}$$

the bucket is emitted without further refinement.
If

$$\mu_b = \text{Descending}$$

the bucket is emitted using reversed ordering.
No MSD, LSD, or tuple processing is performed on monotonic buckets.

Remark 18. *In practical workloads monotonic elimination may remove substantial portions of the input from further refinement, particularly after repeated partitioning reduces local entropy.*

5.9 Full Bucket Descriptor

Each bucket is represented as:

$$D_b = (h_b, \text{OR}_b, \text{AND}_b, V_b, \mu_b)$$

where

$$\mu_b \in \{\text{Ascending}, \text{Descending}, \text{Mixed}\}$$

denotes the bucket monotonic state.

Remark 19. *This descriptor is sufficient to determine all dispatch decisions in A.P.E.X.*

6 Dispatch Hierarchy

A.P.E.X. determines execution using a deterministic bucket-level dispatch system. Each bucket descriptor

$$D_b = (h_b, \text{OR}_b, \text{AND}_b, V_b, \mu_b)$$

is evaluated exactly once per refinement stage to determine the next operation.

6.1 Dispatch Principle

Each bucket is assigned exactly one execution path at each refinement step. The dispatch system is ordered by priority, ensuring deterministic behavior.

6.2 Terminal Conditions

A bucket terminates immediately under the following conditions:

1. **Empty Bucket:** $h_b = 0$
2. **Resolved Bucket:** $V_b = 0$
3. **Monotonic Bucket:**
   - Ascending $\rightarrow$ terminate
   - Descending $\rightarrow$ reverse then terminate

6.3 Non-Terminal Dispatch Rules

If no terminal condition applies, the bucket enters one of the following refinement modes:

1. **Tiny Bucket Sort**

   If:

   $$h_b \leq n_{\text{tiny}}$$

   the bucket is resolved using an implementation-defined comparison-based bridge sorter.

   The reference implementation employs a tiered strategy consisting of insertion sort, binary insertion sort, median-of-three Quicksort [3], Bentley–McIlroy three-way partitioning [18], and a small MSD radix fallback depending on bucket population.

   Remark 20. *The specific bridge-sort policy affects performance but does not affect correctness. Any deterministic sorting procedure producing lexicographic order may be used.*

2. **Tuple Dispatch Condition**

   If:

   $$N_{\text{RVB}}(b) \leq \theta_{\text{tuple}}$$

   Additional implementation-defined saturation criteria may be applied before tuple collapse is selected.

   apply tuple-based refinement over a hardware-bounded projection width of at most 16 bits.

   Tuple Saturation Criterion Tuple collapse is only beneficial when the projected tuple space remains sufficiently occupied.

   Let

   $$m_b = N_{\text{RVB}}(b).$$

   The projected tuple space contains at most

   $$2^{\min(m_b,16)}$$

   distinct states.

   A bucket is eligible for tuple collapse only when the projected state space remains sufficiently saturated relative to bucket population.

   One implementation-specific saturation criterion is

   $$2^{\min(m_b,W_{\max})} \leq h_b.$$

   Remark 21. *This criterion prevents sparse projected state spaces from replacing more efficient MSD or LSD refinement paths.*

3. **MSD Refinement**

   If:

   $$V_b \neq 0$$

   and

   $$D_b \in \mathcal{E}_{\text{MSD}}$$

   apply MSD partitioning over V_b, where

   $$f_{\text{MSD}} : \{0,1\}^{|M_s|} \to \mathbb{N}$$

   is a deterministic threshold function defined solely over the active variable-bit domain.

4. **LSD Refinement**

   If MSD is no longer beneficial, apply packed LSD cycles over the active domain V_b.

6.4 Dispatch Ordering Guarantee

Remark 22 (Dispatch Exclusivity Invariant)**.** *The dispatch system enforces strict priority ordering:*

$$\textit{Terminal} > \textit{Tiny} > \textit{Tuple} > \textit{MSD} > \textit{LSD}$$

Each lower-priority rule is evaluated only if all higher-priority conditions are false.

Theorem 23 (Deterministic Dispatch)**.** *For any bucket descriptor D_b, exactly one dispatch rule applies.*

Proof. Dispatch predicates are evaluated according to a strict priority ordering.

A dispatch rule is considered only if all higher-priority rules evaluate to false. Once a rule is selected, evaluation terminates.

Therefore exactly one execution path is chosen regardless of whether lower-priority predicates would also evaluate to true. Since the priority ordering is total, dispatch selection is deterministic. □

6.5 Key Structural Insight

Remark 24. *Dispatch in A.P.E.X. is not phase-based. Instead, it is a state machine driven by bucket descriptors, where execution path is fully determined by local structure rather than global algorithm stages.*

7 MSD and LSD Refinement Engine

Once a bucket enters a non-terminal state under the dispatch hierarchy, A.P.E.X. performs refinement over the bucket's active variable-bit domain. All refinement is strictly constrained to positions defined by the Variable-Bit Mask V_b.

7.1 Active Refinement Constraint

All refinement operations are restricted to:

$$V_b = (\mathrm{OR}_b \oplus \mathrm{AND}_b) \wedge M_s.$$

Only bit positions satisfying $V_b[j] = 1$ participate in refinement. Constant bit positions are permanently excluded from all subsequent computation.

7.2 MSD Refinement Model

Most-Significant-Digit (MSD) refinement operates over the highest-order structure present in the active variable-bit domain.

Definition 25 (MSD Selection Operator)**.** *Let $\mathcal{A}(b)$ denote the active variable-bit set. The MSD refinement region is defined as the set of highest-order active bits subject to the radix width constraint r:*

$$S_{MSD} \subseteq \mathcal{A}(b)$$

such that:

$$\max(S_{MSD}) \in \mathcal{A}(b)$$

and:

$$|S_{MSD}| \leq r$$

where S_{MSD} consists of the r highest-order elements of $\mathcal{A}(b)$ (or fewer if $|\mathcal{A}(b)| < r$).

Remark 26. *MSD refinement is determined entirely by the structure of V_b, not by fixed radix offsets.*

Definition 27 (MSD Eligibility Function)**.** *MSD eligibility is determined by a deterministic implementation-defined threshold function*

$$f_{\mathrm{MSD}}(D_b)$$

operating on the bucket descriptor

$$D_b = (h_b, \mathrm{OR}_b, \mathrm{AND}_b, V_b, \mu_b)$$

and any implementation-defined workload metrics used by the refinement engine.

Remark 28. *The specific threshold policy affects performance but does not affect correctness. Any deterministic policy preserving dispatch ordering yields identical sorted output.*

Remark 29 (Degenerate Entropy Case)**.** *If no reduction in the Variable-Bit Mask occurs across refinement stages (i.e., $V_b = M_s$ persists), A.P.E.X. degenerates to standard fixed-width MSD radix refinement.*

In this regime, no descriptor compression occurs and performance reduces to classical radix sorting bounds.

7.3 Local MSD Repartitioning

If a bucket remains above structural thresholds, MSD refinement may be recursively applied.

Definition 30 (Local MSD Condition)**.** *A bucket is eligible for local MSD repartitioning only if all enabled implementation-defined thresholds are satisfied.*

These thresholds may include:

- *minimum bucket population $H_{\min}$,*
- *minimum estimated refinement passes $P_{\min}$,*
- *minimum active-window variable-bit density $W_{\min}$,*
- *minimum global workload share $S_{\min}$.*

Only buckets satisfying all enabled thresholds are eligible for local MSD repartitioning.

Remark 31. *Local MSD refinement operates strictly within V_b and does not reintroduce global key-space scanning.*

7.4 Descriptor-Guided Local MSD Repartitioning

Large buckets may remain structurally complex even after an initial MSD partition.

Rather than immediately transitioning to LSD refinement, A.P.E.X. may perform a secondary descriptor-guided MSD repartitioning stage within an existing bucket.

Let

$$D_b = (h_b, \mathrm{OR}_b, \mathrm{AND}_b, V_b, \mu_b)$$

be the descriptor associated with bucket (b).

A bucket is eligible for local MSD repartitioning only if a deterministic eligibility function

$$g_{\mathrm{local}}(D_b)$$

evaluates to true.

The eligibility function may incorporate implementation-defined thresholds including:

$$H_{\min}$$

(minimum bucket population),

$$P_{\min}$$

(minimum estimated refinement passes),

$$W_{\min}$$

(minimum variable-bit density),
and

$$S_{\min}$$

(minimum global workload share).

When eligible, a new MSD partition is constructed using only the active variable-bit domain represented by

$$(V_b)$$

.

The resulting child buckets inherit independent descriptors and are evaluated recursively under the same dispatch hierarchy.

Local MSD repartitioning may additionally serve as a parallel work-generation mechanism.

Large buckets satisfying local eligibility criteria may be repartitioned even when direct refinement would remain correct, allowing unresolved ordering structure to be distributed across multiple worker threads.

Consequently local MSD refinement contributes both to descriptor reduction and workload balancing.

Remark 32. *Local MSD repartitioning operates entirely within the residual variable-bit domain and never reintroduces previously eliminated key positions.*

7.5 LSD Refinement Model

When MSD refinement no longer produces meaningful partition separation, A.P.E.X. transitions to Least-Significant-Digit refinement over the same active domain.

Let:

$$P_b = \{j \mid V_b[j] = 1\}.$$

LSD refinement is executed as a sequence of stable cycles:

$$C_1, C_2, \ldots, C_k$$

such that

$$\bigcup_{m=1}^{k} C_m = P_b$$

and

$$C_i \cap C_j = \varnothing \qquad (i \neq j).$$

The cycles are ordered from least-significant to most-significant active bit positions. Formally,

$$\max(C_m) < \min(C_{m+1}) \qquad \forall m \in \{1, \ldots, k-1\}.$$

Each cycle performs a stable refinement over its assigned bit subset.

7.6 Packed Variable-Bit Cycle Planning

Classical LSD radix sorting operates on contiguous fixed-width windows.

A.P.E.X. instead constructs refinement cycles directly from the active variable-bit domain.

Let

$$P_b = \{j \mid V_b[j] = 1\}.$$

A cycle planner partitions

$$(P_b)$$

into a sequence of refinement cycles

$$C_1, C_2, \ldots, C_k$$

such that

$$\bigcup_i C_i = P_b.$$

The planner may choose either:

1. contiguous active-bit cycles,
2. packed tuple cycles.

Packed tuple cycles may combine multiple active variable-bit positions into a single refinement operation.

When the resulting projected state space remains within implementation-defined limits, several active ordering positions can be resolved during one cycle rather than requiring separate LSD passes.

Remark 33. *This optimization reduces refinement depth by trading additional local state space for fewer global refinement cycles.*

The selected plan minimizes the estimated number of refinement passes while preserving ordering correctness.

Theorem 34 (LSD Stability (Local))**.** *LSD refinement preserves ordering within each MSD-generated bucket.*

Proof. Each LSD cycle is a stable partition over a subset of V_b. Stability is required only within a single bucket domain. Since MSD partitions define disjoint ordering subproblems, no cross-bucket stability preservation is required. Thus LSD stability holds locally. □

Remark 35. *Cycle construction is performed over active variable bits rather than fixed key positions. Consequently LSD refinement cost depends on residual structure rather than physical bit location.*

7.7 Cycle Stability Property

Theorem 36 (LSD Stability)**.** *Each LSD cycle preserves the relative ordering established by prior cycles.*

Proof. Each cycle performs a stable partition over a subset of V_b. Since cycles are executed in increasing bit-significance order and each cycle is stable, previously established lower-significance ordering is preserved. Therefore cumulative refinement maintains lexicographic ordering over the active domain. □

7.8 MSD–LSD Transition Rule

A.P.E.X. dynamically selects refinement mode based on residual structure in the active domain:

- MSD and LSD selection are determined by descriptor state and refinement feasibility tests. MSD is preferred when additional high-order partitioning is expected to reduce residual structure, while LSD is preferred when direct refinement over the active variable-bit domain is more efficient.

Remark 37. *MSD and LSD are not separate phases but adaptive operators acting on the same variable-bit descriptor space.*

7.9 Variable-Bit Relocation

After each refinement step, bucket descriptors are recomputed:

$$V_b = (\mathrm{OR}_b \oplus \mathrm{AND}_b) \wedge M_s.$$

Consequently, active ordering positions may migrate within the key as refinement proceeds.

Remark 38. *A.P.E.X. does not operate on fixed radix windows. Refinement continuously relocates toward the remaining variable-bit structure identified by the current bucket descriptor.*

7.10 Refinement Termination Conditions

Refinement halts when any of the following hold:

- $V_b = 0$ (fully resolved bucket)
- bucket is monotonic under μ_b
- tuple collapse threshold is reached
- bucket size satisfies $h_b \leq n_{\mathrm{tiny}}$

MSD and LSD refinement operate under independent radix width parameters, r and r_{lsd}, which control partition granularity and cycle compression respectively.

8 Tuple Collapse and Residual Domain Projection

When the residual variable-bit structure of a bucket becomes sufficiently small, A.P.E.X. transitions from bitwise refinement to explicit enumeration over the reduced ordering domain defined by V_b. This mechanism eliminates unnecessary refinement cycles while preserving deterministic ordering.

8.1 Tuple Collapse Condition

Let:

$$m_b = N_{\mathrm{RVB}}(b)$$

A bucket enters tuple collapse when:

$$m_b \leq \theta_{\mathrm{tuple}}.$$

Remark 39. *This condition identifies buckets whose remaining ordering structure is sufficiently low-dimensional to permit direct enumeration.*

8.2 Residual Domain Projection

Each key is projected onto the active variable-bit domain:

Definition 40 (Domain Projection)**.** *For each key k, define*

$$\pi_b(k) = (k_{j_1}, k_{j_2}, \ldots, k_{j_m})$$

where

$$j_1 < j_2 < \cdots < j_m$$

are the elements of $\mathcal{A}(b)$.

This projection removes all constant bit positions and retains only those bits that influence ordering within the bucket.

8.3 Projected Ordering Space

The projected space has cardinality bounded by:

$$|\pi_b(R_b)| \leq 2^{m_b}.$$

Remark 41. *This bound reflects the maximal distinguishable configurations over the active variable-bit domain.*

8.4 Tuple-Based Resolution

Once projected, ordering is performed over the reduced domain using direct enumeration or counting-based sorting over $\pi_b(R_b)$.

Remark 42. *Tuple resolution replaces iterative bitwise refinement with a single deterministic pass over the reduced state space.*

8.5 Correctness of Projection

Theorem 43 (Projection Ordering Equivalence)**.** *Ordering in the projected domain is equivalent to ordering in the original domain restricted to $\mathcal{A}(b)$.*

Proof. All constant bit positions are identical across all elements of R_b and therefore do not affect comparisons between keys. The projection π_b preserves both the values and the positional ordering of all variable bit positions contained in $\mathcal{A}(b)$. Since these positions contain all remaining ordering information within the bucket, lexicographic ordering is preserved under π_b. □

8.6 Hardware-Bounded Tuple Projection

In practice, tuple projection width is bounded by an implementation-defined limit

$$m_b \leq W_{\max}.$$

For the implementation evaluated in this paper,

$$W_{\max} = 16.$$

Consequently tuple-space cardinality satisfies

$$|\pi_b(R_b)| \leq 2^{16}.$$

This bound guarantees finite counting-space allocation and predictable memory requirements during tuple collapse.

Remark 44. *The tuple bound is an implementation constraint rather than a theoretical limitation of the descriptor model.*

The computational cost of tuple-based resolution is:

$$T_{\text{tuple}}(b) = \mathcal{O}\Big(h_b + 2^{\min(m_b,16)}\Big).$$

Remark 45. *This replaces repeated refinement cycles with a single bounded evaluation over the reduced domain.*

8.7 Interaction with Refinement Engine

Tuple collapse interrupts MSD and LSD refinement when:

- residual variable-bit count satisfies $m_b \leq \theta_{\text{tuple}}$,
- or further refinement does not reduce V_b.

8.8 Key Structural Insight

Remark 46. *Tuple collapse is a deterministic transition from bitwise refinement space to explicit enumeration over the residual ordering domain defined by V_b.*

8.9 Role in Dispatch Hierarchy

Tuple collapse constitutes the final refinement stage in the A.P.E.X. execution model:

1. MSD refinement (high-order structure over V_b)
2. LSD refinement (distributed structure over V_b)
3. Tuple collapse (explicit resolution over V_b)

Remark 47. *This hierarchy ensures that refinement operates over the remaining variable-bit structure rather than the full key width. Runtime then depends on the selected refinement operator, bucket population, and size of the active domain.*

9 Parallel Execution Model

A.P.E.X. is designed for deterministic shared-memory parallel execution. The algorithm decomposes computation into four stages: thread-local histogram construction, descriptor reduction, deterministic layout computation, and scatter execution.

Assumption 48 (Thread Segment Contiguity)**.** *The input sequence is partitioned into contiguous segments assigned to threads.*

Each thread operates on a continuous range of the original input array without interleaving elements from other threads.

9.1 Thread-Local Histogram Construction

Let p be the number of threads. The input sequence R is partitioned into p disjoint segments.

Each thread t computes local bucket statistics:

$$h_{b,t}, \quad \mathrm{OR}_{b,t}, \quad \mathrm{AND}_{b,t}.$$

- $h_{b,t}$: local bucket population count
- $\mathrm{OR}_{b,t}$: bitwise OR accumulator
- $\mathrm{AND}_{b,t}$: bitwise AND accumulator

No synchronization is required during this phase.

9.2 Descriptor Reduction

Global bucket descriptors are computed via associative reduction:

$$h_b = \sum_{t=1}^{p} h_{b,t}$$

$$\mathrm{OR}_b = \bigvee_{t=1}^{p} \mathrm{OR}_{b,t}$$

$$\mathrm{AND}_b = \bigwedge_{t=1}^{p} \mathrm{AND}_{b,t}$$

Remark 49. *Bitwise OR and AND operations are associative and commutative, ensuring equivalence between parallel and sequential aggregation.*

9.3 Global Monotonic State Reduction

Let each thread t operate on a contiguous slice $R_{b,t}$.

Each slice has:

$$S_{b,t} \in \{\text{Ascending}, \text{Descending}, \text{Mixed}, \text{Empty}\}$$

and boundary extrema:

$$\min_{b,t}, \quad \max_{b,t}$$

For empty slices, $\min_{b,t}$ and $\max_{b,t}$ are undefined and do not participate in boundary comparisons.

Let

$$T_b = \{\, t \mid h_{b,t} > 0 \,\}$$

denote the ordered set of non-empty thread partitions associated with bucket b.

9.3.1 Local Monotonic Classification

For a non-empty thread-local slice

$$R_{b,t} = (k_1, k_2, \ldots, k_m),$$

the local state is defined as:

$$S_{b,t} = \text{Ascending}$$

if

$$k_i \leq k_{i+1} \qquad \forall i \in \{1, \ldots, m-1\}.$$

Similarly,

$$S_{b,t} = \text{Descending}$$

if

$$k_i \geq k_{i+1} \qquad \forall i \in \{1, \ldots, m-1\}.$$

If neither condition holds,

$$S_{b,t} = \text{Mixed}.$$

Empty slices are assigned

$$S_{b,t} = \text{Empty}.$$

9.3.2 Local Validity Constraint

A slice is eligible for global monotonic inference only if:

$$S_{b,t} \in \{\text{Ascending}, \text{Empty}\} \quad \forall t$$

or:

$$S_{b,t} \in \{\text{Descending}, \text{Empty}\} \quad \forall t$$

Mixed slices invalidate global monotonic inference.

9.3.3 Global Ascending Condition

$$\text{Global Ascending} \iff (\forall t,\ S_{b,t} \in \{\text{Ascending}, \text{Empty}\}) \wedge \left(\forall i \in [1, |T_b| - 1], \quad \max_{b,t_i} \leq \min_{b,t_{i+1}}\right)$$

If the condition holds, then

$$\mu_b = \text{Ascending}.$$

9.3.4 Global Descending Condition

$$\text{Global Descending} \iff (\forall t,\ S_{b,t} \in \{\text{Descending}, \text{Empty}\}) \wedge \left(\forall i \in [1, |T_b| - 1], \quad \min_{b,t_i} \geq \max_{b,t_{i+1}}\right)$$

If the condition holds, then

$$\mu_b = \text{Descending}.$$

Theorem 50 (Local-to-Global Monotonicity)**.** *Let $T_b = (t_1, t_2, \ldots, t_q)$ be the ordered set of non-empty thread-local slices for bucket b, preserving input order. All comparisons are taken under the unsigned key order. If every non-empty slice is locally ascending and*

$$\max_{b,t_i} \leq \min_{b,t_{i+1}} \qquad \forall i \in \{1, \ldots, q-1\},$$

then R_b is globally ascending. If every non-empty slice is locally descending and

$$\min_{b,t_i} \geq \max_{b,t_{i+1}} \qquad \forall i \in \{1, \ldots, q-1\},$$

then R_b is globally descending.

Proof. If $q \leq 1$, the result follows directly from the local classification of the only non-empty slice, or vacuously for an empty bucket. Assume $q > 1$.

Consider the ascending case. Within each non-empty slice R_{b,t_i}, local ascending classification gives

$$k_a \leq k_{a+1}$$

for all adjacent records inside that slice. The boundary condition gives

$$\max_{b,t_i} \leq \min_{b,t_{i+1}}$$

for each consecutive pair of non-empty slices in input order. Since a locally ascending slice has its minimum at the first key and its maximum at the last key, the last key of R_{b,t_i} is no greater than the first key of $R_{b,t_{i+1}}$. Combining the internal adjacent inequalities with these cross-slice boundary inequalities shows that every adjacent pair in the concatenated bucket sequence R_b is nondecreasing. Hence R_b is globally ascending.

The descending case is symmetric: local descending classification gives nonincreasing order inside each slice. Since a locally descending slice has its maximum at the first key and its minimum at the last key, the boundary condition $\min_{b,t_i} \geq \max_{b,t_{i+1}}$ ensures that the last key of one slice is no smaller than the first key of the next slice. Thus every adjacent pair in the concatenated bucket sequence is nonincreasing, so R_b is globally descending.

Empty slices are omitted from T_b, so they introduce no adjacent records and do not affect the argument. □

9.3.5 Otherwise

$$\mu_b = \text{Mixed}$$

Remark 51. *Boundary monotonicity alone is insufficient. Global monotonic inference requires both internal slice consistency and cross-slice boundary ordering.*

9.4 Variable-Bit Mask Reconstruction

After reduction, the Variable-Bit Mask is computed as:

$$V_b = (\mathrm{OR}_b \oplus \mathrm{AND}_b) \wedge M_s.$$

This step completes descriptor formation for all buckets.

9.5 Deterministic Layout Construction

A prefix sum over bucket populations defines the global memory layout, following the standard use of prefix sums for parallel placement and compaction operations [6, 5, 4]:

$$\mathrm{offset}_b = \sum_{i<b} h_i.$$

Theorem 52 (Deterministic Layout Property)**.** *Given a fixed input sequence, radix configuration, and thread count, the output layout is uniquely determined.*

Proof. Bucket populations h_b are deterministic functions of the input. Prefix sums over deterministic values produce a unique, non-overlapping memory layout. Therefore all write offsets are fixed prior to scatter execution. □

9.6 Scatter Phase

Each record is written exactly once into its assigned output position using precomputed offsets.

Remark 53. *Scatter operations are write-disjoint across threads and require no synchronization.*

9.7 Adaptive Work Distribution

Refinement tasks are represented as independent bucket-level work units.

Work units may correspond to:

- MSD buckets,
- local MSD partitions,
- LSD refinement tasks,
- tuple-collapse tasks.

Workers dynamically acquire tasks from a shared work queue.

Large tasks may be processed independently while smaller tasks are grouped into batches to reduce scheduling overhead.

Remark 54. *Dynamic work acquisition improves load balancing under highly skewed distributions where bucket populations differ substantially.*

9.8 LSD Work Stealing

LSD refinement tasks may be dynamically redistributed between workers when substantial load imbalance exists.

Work stealing operates only on independent refinement tasks and therefore preserves deterministic output.

Remark 55. *Work stealing affects execution scheduling but does not modify dispatch decisions or ordering correctness.*

9.9 Descending Bucket Emission

Buckets classified as Descending are emitted using an inverted output mapping.

Let the bucket occupy output positions

$$[\text{offset}_b,\ \text{offset}_b + h_b - 1].$$

For descending buckets, let

$$r_t \in [0, h_{b,t} - 1]$$

denote the thread-local rank of a record within thread t's contribution to bucket b.

The destination position is

$$\text{pos}(r_t) = \text{offset}_b + h_b - 1 - \text{offset}_{b,t} - r_t.$$

Thus records are written in reverse order within the bucket region while preserving deterministic ownership of thread-local subranges.

Remark 56. *Descending termination is therefore implemented as an inverted scatter operation rather than an explicit post-processing reversal pass.*

9.10 Thread-Local Offset Computation

Within each bucket, thread-local offsets are defined as:

$$\text{offset}_{b,t} = \sum_{i<t} h_{b,i}.$$

This ensures each thread writes into a unique subrange of the bucket region.

9.11 Correctness of Parallel Execution

Theorem 57 (Scatter Determinism)**.** *Refinement stages may alternate between source and destination buffers.*

Successive refinement cycles therefore operate over alternating memory regions without requiring in-place permutation.

Remark 58. *Alternating scatter buffers simplify parallel execution and eliminate cycle-following operations commonly required by in-place radix variants.*

Each record is written exactly once to a unique index in the output array.

Proof. Each record is assigned a unique bucket via radix partitioning. Within each bucket, thread ownership is uniquely determined by input segmentation. Prefix sums guarantee non-overlapping write regions. Therefore no collisions or duplicate writes occur. □

9.12 Reduction Correctness

Theorem 59 (Descriptor Reduction Equivalence)**.** *Parallel reduction of OR, AND, and population counts is equivalent to sequential computation.*

Proof. Summation, bitwise OR, and bitwise AND are associative operations over disjoint partitions. Therefore reduction over thread partitions yields identical results to sequential evaluation over the full input. □

9.13 Key Structural Insight

Remark 60. *Parallelism in A.P.E.X. is achieved by separating computation into independent local accumulation, deterministic reduction, and fixed-layout scatter phases, eliminating runtime synchronization during execution.*

10 Descriptor Reduction Principle

A.P.E.X. may be viewed as a descriptor-driven decomposition system.

Initially, a bucket may contain up to

$$64$$

potential ordering positions. The bucket descriptor

$$D_b = (h_b, \mathrm{OR}_b, \mathrm{AND}_b, V_b, \mu_b)$$

identifies which of those positions remain relevant to ordering.

The residual variable-bit count is not, by itself, used as a strict ranking function for all execution states. In particular, a child bucket may remain saturated relative to its current active mask, and consecutive descriptors may therefore have the same N_{RVB} value. The formal progress argument must account for the full execution state, including the bucket range, active mask, and pending refinement schedule.

Theorem 61 (Descriptor Decomposition)**.** *For every non-terminal bucket b, execution of a dispatch-selected refinement operator produces either*

1. *a terminal descriptor state, or*
2. *a finite collection of successor descriptor states whose record ranges form a disjoint decomposition of R_b.*

This statement does not assert that N_{RVB} strictly decreases for every successor descriptor.

Proof. Terminal dispatch rules immediately halt refinement.

Otherwise, MSD refinement partitions the bucket into a finite set of sub-buckets whose ordering structure is represented by new descriptors. LSD refinement applies a finite cycle plan over the active variable-bit domain. Tuple collapse replaces residual refinement with direct enumeration over the projected ordering space.

Thus a non-terminal refinement step either produces a terminal descriptor or replaces the parent bucket by a finite set of successor subproblems. The construction does not require the scalar value N_{RVB} to strictly decrease at every descriptor transition.

□

Theorem 62 (No Descriptor-Cycle Execution)**.** *Along any single execution path, A.P.E.X. cannot repeatedly reprocess the same bucket range with the same active refinement state indefinitely.*

Proof. The descriptor D_b is only part of the execution state. A work item also includes the record range being processed, the active refinement mask, and the remaining refinement schedule selected by dispatch.

Terminal, monotonic, tiny-sort, and tuple-collapse paths halt for their bucket. LSD refinement is planned as a finite sequence of disjoint cycles over $P_b = \{j \mid V_b[j] = 1\}$; each executed cycle consumes one element of that finite schedule and is not reintroduced.

For MSD or local-MSD refinement, the selected radix positions become part of the child bucket identity. The parent range is not placed back on the work queue as the same unresolved state; it is replaced by finite child ranges whose descriptors are evaluated as new subproblems under the dispatch hierarchy.

Therefore a repeated numerical value of N_{RVB} does not imply an execution cycle. Even in the degenerate case where descriptors remain saturated relative to the current active mask, execution follows the finite fixed-width refinement schedule rather than revisiting the same full state indefinitely.

□

Remark 63. *The primary objective of A.P.E.X. is not to prove that every descriptor update strictly decreases N_{RVB}. The objective is to eliminate or decompose unresolved ordering structure using deterministic operators whose execution states cannot cycle.*

10.1 Descriptor-Driven Cycle Elimination

The practical objective of A.P.E.X. is not to avoid reading the input, but to avoid refinement passes over bit positions that cannot affect ordering inside the current bucket.

Let

$$m_b = N_{\mathrm{RVB}}(b).$$

Dispatch-selected refinement may:

1. reduce bucket population through decomposition,
2. reduce residual variable-bit count,
3. establish monotonic or resolved structure,
4. consume a finite planned refinement cycle,
5. or reach tuple collapse.

The scalar value m_b need not decrease after every descriptor transition. Nevertheless, refinement plans are constructed over the active domain $\mathcal{A}(b)$, so the number of eligible bit-refinement cycles is tied to the residual variable-bit structure rather than to all physical key positions.

Remark 64. *The implementation therefore behaves as a descriptor-guided refinement engine rather than a conventional fixed-schedule radix engine. Radix refinement is one mechanism used to eliminate unresolved descriptor structure.*

11 Complexity Analysis in Variable-Bit Space

The complexity model separates mandatory record traversal from descriptor-guided refinement. A.P.E.X. still reads the input records during descriptor construction, and selected execution paths may scatter or copy all records. The residual variable-bit measure does not remove this linear component. Instead, it describes the structure that controls how much additional refinement is scheduled after descriptors are known.

11.1 Notation

Let:

- n: number of records,
- p: number of threads,
- h_b: bucket population,
- $m_b = N_{\mathrm{RVB}}(b)$: residual variable-bit count,
- $\mathcal{F}$: a frontier of active bucket descriptors at a given refinement stage.

Definition 65 (Residual Variable-Bit Potential)**.** *For a bucket frontier $\mathcal{F}$, define*

$$\Phi(\mathcal{F}) = \sum_{b \in \mathcal{F}} N_{\mathrm{RVB}}(b) = \sum_{b \in \mathcal{F}} m_b.$$

Φ is an unweighted structural potential: it counts the total number of active ordering bit positions present across the current descriptor frontier. It is not, by itself, a runtime bound because record movement also depends on bucket populations h_b.

11.2 Global Phases

Histogram construction and scatter phases operate over the full input in parallel. Their total work remains linear in the number of records.

$$W_{\text{hist}} = \Theta(n), \qquad T_{\text{hist}} = O\left(\frac{n}{p}\right)$$

$$W_{\text{scatter}} = O(n), \qquad T_{\text{scatter}} = O\left(\frac{n}{p}\right)$$

Remark 66. *Both histogram construction and scatter phases are implemented as thread-local accumulation followed by deterministic parallel reduction or write phases. Therefore, their effective parallel time scales with $\frac{n}{p}$ under ideal work distribution, but their total work remains linear. The descriptor model affects later refinement scheduling; it does not avoid the initial full-input read.*

11.3 Refinement Cost Structure

Refinement cost is determined independently per bucket based on bucket population h_b, dispatch choice, and active-domain size m_b. The role of m_b is to control the number of active bit positions, refinement cycles, or tuple states considered by the selected operator.

11.3.1 Monotonic Termination

If a bucket is monotonic under μ_b:

$$T_{\text{mono}}(b) = \mathcal{O}(h_b)$$

11.3.2 Resolved Buckets

If:

$$V_b = 0$$

then no further refinement is required.

$$T_{\text{resolved}}(b) = \mathcal{O}(1)$$

beyond descriptor construction.

11.3.3 MSD Refinement

Each MSD refinement level processes the records in the bucket once and partitions them according to selected active bit positions. If ρ_b MSD or local-MSD levels are applied to bucket b, then:

$$T_{\text{MSD}}(b) = \mathcal{O}(h_b \cdot \rho_b).$$

The value ρ_b is determined by dispatch thresholds, bucket population, and the finite active refinement state. It is not asserted to be equal to m_b, and in saturated or degenerate cases MSD refinement may follow the classical fixed-width schedule.

11.3.4 LSD Refinement

For packed LSD cycles over the active domain, the number of cycles is proportional to the number of residual variable bits:

$$C_b \leq \left\lceil \frac{m_b}{r_{\text{lsd}}} \right\rceil$$

$$T_{\text{LSD}}(b) = \mathcal{O}(h_b \cdot C_b)$$

The factor h_b remains because each selected cycle processes the records in the bucket. Thus RVB reduces the number of cycles considered, not the need to visit records participating in a cycle.

11.3.5 Tuple Collapse

When:

$$m_b \leq \theta_{\text{tuple}}$$

refinement becomes:

$$T_{\text{tuple}}(b) = \mathcal{O}(h_b + 2^{\min(m_b, W_{\max})})$$

where $W_{\max} \leq 16$ in the evaluated implementation.

11.3.6 Tiny Bucket Case

For small buckets:

$$h_b \leq n_{\text{tiny}}$$

$$T_{\text{tiny}}(b) = \mathcal{O}(h_b \log h_b)$$

11.4 Frontier Potential and Runtime Model

Let $B^+ = \{b \mid h_b > 0\}$

Remark 67 (Dynamic Bucket Domain)**.** *The set B^+ denotes the collection of all non-empty buckets generated during execution, including recursively produced MSD and LSD child buckets.*

It is not restricted to the initial radix index space $[0, 2^r - 1]$, which applies only to the root partitioning stage.

At any frontier $\mathcal{F} \subseteq B^+$, $\Phi(\mathcal{F})$ summarizes the unresolved structural width present across active buckets. For example, if all buckets in $\mathcal{F}$ enter LSD refinement with width r_{lsd}, the number of scheduled LSD cycles across the frontier is bounded by

$$\sum_{b \in \mathcal{F}} \left\lceil \frac{m_b}{r_{\text{lsd}}} \right\rceil \leq |\mathcal{F}| + \frac{\Phi(\mathcal{F})}{r_{\text{lsd}}}.$$

This is the intended role of Φ: it measures refinement structure. The record work for those cycles is still population-weighted:

$$W_{\text{LSD}}(\mathcal{F}) = \mathcal{O}\left(\sum_{b \in \mathcal{F}} h_b \left\lceil \frac{m_b}{r_{\text{lsd}}} \right\rceil \right).$$

Thus an aggregate work model is:

$$W(n) = \Theta(n) + \sum_{b \in B^+} W_{\text{refine}}(b)$$

where the leading term represents mandatory full-input traversal phases, and $W_{\text{refine}}(b)$ is defined by the deterministic dispatch mapping. Let ρ_b denote the number of recursive MSD refinement levels applied to bucket b.

$$W_{\text{refine}}(b) = \begin{cases} O(h_b) & \text{if monotonic or resolved termination applies} \\ O(h_b \log h_b) & \text{if tiny bucket rule applies} \\ O(h_b \cdot \rho_b) & \text{if MSD refinement applies} \\ O(h_b \cdot C_b) & \text{if LSD refinement applies} \\ O(h_b + 2^{\min(N_{\text{RVB}}(b), W_{\max})}) & \text{if tuple collapse applies} \end{cases}$$

where C_b denotes the number of deterministic refinement cycles scheduled over the active variable-bit domain V_b, and $N_{\text{RVB}}(b)$ is the residual variable-bit count. Parallel time depends on how this work is distributed across workers, with the mandatory linear phases contributing $\mathcal{O}(n/p)$ under ideal balance.

11.5 Key Theorem

Theorem 68 (Residual-Domain Refinement)**.** *For any bucket b, all refinement operations performed after descriptor construction are restricted to the active variable-bit domain*

$$\mathcal{A}(b) = \{j \mid V_b[j] = 1\},$$

whose cardinality is

$$|\mathcal{A}(b)| = N_{\text{RVB}}(b).$$

Bit positions outside $\mathcal{A}(b)$ do not participate in subsequent refinement decisions.

Proof. By the Variable-Bit Characterization Lemma, the active domain

$$\mathcal{A}(b)$$

contains exactly those bit positions exhibiting variation within bucket b.

All refinement operators are defined exclusively over V_b or equivalently $\mathcal{A}(b)$.

Bit positions outside $\mathcal{A}(b)$ are constant over the bucket and therefore never participate in subsequent ordering operations.

Consequently descriptor-guided refinement excludes all positions outside

$$\mathcal{A}(b),$$

rather than operating over the full remaining key width.

□

Remark 69 (MSD-to-LSD Transition Semantics)**.** *The MSD-to-LSD transition is a deterministic mode selection applied at the level of a single bucket descriptor.*

A bucket is assigned either MSD refinement or LSD refinement based on its descriptor state prior to any refinement.

Once a bucket enters LSD mode, it is treated as an independent ordering domain and is never reinterpreted as part of a higher-level MSD partition.

LSD refinement does not depend on global stability across MSD partitions; instead, it operates strictly within the isolated bucket subproblem defined at the moment of MSD completion.

Remark 70 (Runtime Implication)**.** *The preceding theorem is a domain restriction, not a complete runtime bound. The runtime bounds above also depend on bucket population h_b, dispatch choice, MSD recursion depth ρ_b, LSD cycle count C_b, and the tuple projection cap. Reducing $N_{\text{RVB}}(b)$ can reduce the number of eligible refinement cycles or tuple states, but the theorem itself establishes only that constant bit positions are excluded from subsequent refinement.*

11.6 Fixed-Width Specialization

For 64-bit keys,

$$m_b \leq 64.$$

Therefore the refinement space is always finite.

Remark 71. *The practical significance of A.P.E.X. is not the finite upper bound itself, but that refinement is restricted to the residual variable-bit structure of each bucket rather than requiring uniform examination of all 64 key positions.*

11.7 Structural Insight

Remark 72. *A.P.E.X. operates as a descriptor-reduction system in which refinement decisions are organized around unresolved variable-bit structure rather than fixed key width.*

11.8 Space Complexity Model

The A.P.E.X. implementation has a linear record-buffer footprint plus configuration-bounded planning and scratch structures. Let

$$B_{\text{msd}} = 2^r, \qquad R_{\text{lsd}} = 2^{r_{\text{lsd}}}, \qquad R_{\text{tuple}} = 2^{W_{\max}},$$

where $W_{\max} \leq 16$ is the maximum direct tuple projection width. Let H_{scratch} denote the configured per-worker heap scratch record cap, and let C_{local} denote the configured cap on local-MSD child buckets represented by the planner.

$$\begin{aligned} S(n,p) =& \mathcal{O}(n) + \mathcal{O}(pB_{\text{msd}}) + \mathcal{O}(B_{\text{msd}}) \\ &+ \mathcal{O}(p \max(R_{\text{lsd}}, R_{\text{tuple}})) + \mathcal{O}(pH_{\text{scratch}}) + \mathcal{O}(pC_{\text{local}}). \end{aligned} \tag{1}$$

This allocation footprint is derived from the following components:

1. **Primary Record Buffers:** A.P.E.X. stores records as 16-byte key/value pairs. Large runs may hold both a source segment and a destination segment, so the dominant record-buffer footprint is $\mathcal{O}(n)$, approximately $2 \cdot 16n$ bytes in the full-buffer case. The destination segment is allocated lazily when the selected execution path requires it.

2. **Thread-Local Histogram and Descriptor Memory:** The MSD descriptor pass maintains per-thread bucket populations, extremal accumulators, monotonicity descriptors, and scatter offsets over $B_{\text{msd}} = 2^r$ top-level buckets. This contributes $\mathcal{O}(pB_{\text{msd}})$ storage, independent of the dataset scale n.

3. **Global Planning Metadata:** The bucket plan stores starts, sizes, flags, variable masks, LSD cycle counts, tuple-tail masks, and cycle-plan references per MSD bucket, contributing $\mathcal{O}(B_{\text{msd}})$. Optional local-MSD repartitioning adds child starts, sizes, masks, and per-thread child offsets bounded by the configured local-child cap, contributing $\mathcal{O}(pC_{\text{local}})$.

4. **Worker Scratch and Counting Arrays:** Each worker owns bounded counting arrays sized by the larger of the configured LSD radix and direct tuple radix cap, giving $\mathcal{O}(p \max(R_{\text{lsd}}, R_{\text{tuple}}))$. Heap record scratch is bounded by the configured per-worker scratch limit, contributing $\mathcal{O}(pH_{\text{scratch}})$. Tuple collapse therefore does not allocate unbounded 2^{m_b} space; its direct enumeration is capped by $W_{\max} \leq 16$.

Remark 73. *For a fixed A.P.E.X. configuration, B_{msd}, R_{lsd}, R_{tuple}, H_{scratch}, and C_{local} are bounded independently of n. Thus total working memory is $\mathcal{O}(n)$, with the linear term dominated by the source/destination record segments. If the source segment is treated as the input rather than auxiliary storage, the extra space remains $\mathcal{O}(n)$, dominated by the optional destination/scratch record segment.*

12 Global Correctness of A.P.E.X.

A.P.E.X. defines a deterministic sorting system composed of MSD partitioning, variable-bit refinement, monotonic classification, LSD refinement, and tuple collapse. This section establishes that all execution paths preserve unsigned lexicographic ordering.

12.1 Correctness Objective

Let R be the input sequence and R' the output produced by A.P.E.X. The correctness condition is:

$$k_{\sigma(1)} \leq k_{\sigma(2)} \leq \cdots \leq k_{\sigma(n)}.$$

12.2 Execution Path Partitioning

Each record follows exactly one deterministic execution path:

1. Monotonic termination
2. Resolved bucket termination ($V_b = 0$)
3. Tiny bucket sorting
4. MSD refinement path
5. Local MSD refinement path
6. LSD refinement path
7. Tuple collapse path

Remark 74. *These paths are mutually exclusive and collectively exhaustive over all bucket states.*

12.3 MSD Correctness

Theorem 75 (MSD Preservation)**.** *MSD partitioning preserves unsigned lexicographic ordering.*

Proof. MSD refinement partitions records according to the highest-order unresolved bit positions contained in the active variable-bit domain V_b.

For any two keys differing within the selected MSD region, the partition index preserves the ordering induced by the most significant differing active bit.

Keys assigned to different MSD partitions therefore satisfy lexicographic ordering across partitions.

Keys assigned to the same partition remain candidates for further refinement and are processed recursively.

Thus MSD refinement preserves unsigned lexicographic ordering. □

12.4 LSD Correctness

Theorem 76 (LSD Stability)**.** *Packed LSD refinement preserves ordering within each bucket.*

Proof. Each LSD cycle performs stable refinement over a subset of the variable-bit domain V_b. Stability ensures previously established ordering is preserved. Since cycles operate over disjoint subsets of V_b, cumulative refinement maintains global ordering consistency. □

12.5 Tuple Collapse Correctness

Theorem 77 (Projection Equivalence)**.** *Tuple projection preserves lexicographic ordering over the active domain $\mathcal{A}(b)$.*

Proof. Constant bit positions are identical across all elements of R_b and therefore do not affect ordering. The projection π_b retains only variable positions defined by V_b, which fully determine ordering within the bucket. Therefore ordering is preserved under projection. □

12.6 Monotonic Correctness

Theorem 78 (Monotonic Termination)**.** *If a bucket is monotonic, it requires no further refinement.*

Proof. An ascending bucket already satisfies lexicographic ordering. A descending bucket becomes sorted under the inverted scatter mapping defined for descending emission. □

12.7 Resolved Bucket Correctness

Theorem 79 (Active-Domain Resolution)**.** *If*

$$V_b = 0,$$

then no unresolved ordering bits remain within the active refinement domain.

Proof. The mask V_b is computed over the active refinement domain defined by M_s. If $V_b = 0$, every bit position participating in the current refinement stage is constant across the bucket. Therefore no additional refinement within the active domain can affect ordering, and the bucket is resolved with respect to the remaining execution state. □

12.8 Global Correctness Theorem

Theorem 80 (A.P.E.X. Sorting Correctness)**.** *A.P.E.X. produces a correctly sorted sequence for all valid inputs.*

Proof. By the Deterministic Dispatch Theorem, every bucket is assigned exactly one execution path.

The MSD Preservation Theorem, LSD Stability Theorem, Projection Equivalence Theorem, Monotonic Termination Theorem, and Active-Domain Resolution Theorem establish that every possible execution path preserves unsigned lexicographic ordering.

Since the dispatch hierarchy is deterministic and collectively exhaustive, every record is processed by exactly one correctness-preserving operator.

Therefore the final emitted sequence is globally sorted.

□

12.9 Key Structural Insight

Remark 81. *Correctness in A.P.E.X. is derived from deterministic partitioning and refinement over the active variable-bit domain, ensuring that no execution path violates lexicographic ordering.*

13 Experimental Evaluation

This section evaluates A.P.E.X. across a diverse collection of input distributions, problem sizes, and comparison algorithms.

The objective is to measure practical performance, scalability, and robustness under varying entropy and ordering structures.

All experiments were executed on two processor generations to evaluate both architectural scaling and algorithmic behavior.

13.1 Hardware Platforms

Experiments were conducted on two AMD Zen-generation systems using JDK 25.

- **AMD Ryzen 7 1800X System**
  - CPU: AMD Ryzen 7 1800X
  - Cores/Threads: 8C / 16T

  - Frequency: 3.5 GHz
  - Memory: 32 GB DDR4-3200
  - Motherboard: ASUS Crosshair VI Hero
  - Operating System: Windows 10
  - Runtime: JDK 25
- **AMD Ryzen 9 7950X System**
  - CPU: AMD Ryzen 9 7950X
  - Cores/Threads: 16C / 32T
  - Frequency: 4.5 GHz
  - Memory: 64 GB DDR5-5600
  - Motherboard: ASUS TUF X870E- Pro WiFi
  - Operating System: Windows 11
  - Runtime: JDK 25
- **JVM Arguments:**
  - `-enable-native-access=ALL-UNNAMED`
  - `-enable-preview`
  - `-XX:MaxDirectMemorySize=16g` on Ryzen 1800X; `-XX:MaxDirectMemorySize=40G` on Ryzen 7950X
  - `-Xmx8G`
  - `-XX:+UseLargePages` requested
  - `-XX:LargePageSizeInBytes=2m` requested
  - `-add-modules=jdk.incubator.vector` [16]
- **A.P.E.X. Runtime Configuration:**
  - `TupleBits = 16`
  - `tuplePacking = true`
  - `curated = true`
  - `config = 12,13,128`
  - `threads = 16` on Ryzen 1800X; `threads = 32` on Ryzen 7950X
  - `signed = false`
  - `workStealing = true`
  - `workBatch = 8` on Ryzen 1800X; `workBatch = 16` on Ryzen 7950X
  - `directTupleScratchFinal = true`
  - `localMsd = true`

The Ryzen 1800X platform represents a first-generation Zen architecture, while the Ryzen 7950X platform represents a modern Zen 4 architecture. Evaluating both systems provides insight into the scalability of A.P.E.X. across substantially different processor generations and memory subsystems.

Platform	Java runtime	VM	Threads	Direct memory
Ryzen 1800X	`25+36-3489`	OpenJDK 64-Bit Server VM	16	`16g`
Ryzen 7950X	`25.0.3+9-LTS-195`	Java HotSpot 64-Bit Server VM	32	`40G`

Table 1: Exact runtime provenance reported by the comparison logs. Both runs used 21 measured runs after 3 warmups, `-Xmx8G`, preview features, requested large pages, and `jdk.incubator.vector`.

13.2 Benchmark Methodology

Each plotted point reports median throughput across measured runs after warmup runs. Unless otherwise noted, the comparison harness default is 21 measured runs after 3 warmup runs. The harness also records mean, standard deviation, coefficient of variation, p95, and p99 timings; however, the figures in this paper plot only medians and do not include error bars or confidence intervals.

A.P.E.X. used configuration `MSD=12`, `LSD=13`, `TINY=128`, `tupleBits=16`, and `tuplePacking=true`. The `apex-records` series sorts 16-byte key/value records, while `apex-keys`, JDK, Fastutil, and the external IPS^{4}o conversion are key-only unless otherwise stated. The implementation and benchmark harness are available with the source artifact [19].

The IPS^{4}o conversion reported here was run on the Ryzen 1800X system against the same generated key database used for the A.P.E.X. runs. It was compiled as:

```
g++ -O3 -std=c++17 -fopenmp -pthread -march=native

unified_bench.cpp -latomic -o unified_bench.
```

Series	Workload kind	Implementation
`apex-records`	key/value records	A.P.E.X. record pipeline
`apex-keys`	key-only	A.P.E.X. key-only path
JDK parallel sort	key-only	JDK 25 `Arrays.parallelSort`
Fastutil parallel radix	key-only	Fastutil 8.5.13 parallel radix
IPS^{4}o conversion	key-only	external comparison-based sample sort

Table 2: Comparison series used in the throughput plots and external baseline tables. Key-only baselines move less data than the `apex-records` path and should not be interpreted as identical-workload speedups.

Distribution	10^6 rec.	10^7 rec.	10^8 rec.
Random	122.55	241.88	214.86
Low bits only	120.73	166.33	208.78
High bits only	178.52	180.21	216.84
Duplicates	135.19	179.15	264.89
Zipfian	187.08	223.62	201.13
Sparse entropy	175.59	405.25	550.18

Table 3: Median throughput of the external IPS^{4}o key-only conversion on the Ryzen 1800X system, in million records per second. The conversion sorts keys only and is therefore not an identical-workload comparison to `apex-records`. Complete min/median/standard-deviation/CV/p95/p99 statistics are provided in the supplementary `ips4o-baseline.csv` artifact.

Remark 82 (Statistical Scope)**.** *The present figures are median-throughput plots. They are useful engineering evidence, but a stronger experimental section should also plot variability using error bars or confidence intervals derived from the measured runs.*

Platform	Series	CV min/median/max (%)	Max $p95$/median
Ryzen 1800X	`apex-records`	1.31 / 5.91 / 29.35	2.04
Ryzen 1800X	`apex-keys`	2.71 / 4.83 / 8.00	1.19
Ryzen 1800X	JDK parallel sort	1.66 / 3.30 / 12.79	1.20
Ryzen 1800X	Fastutil parallel radix	1.43 / 3.28 / 7.94	1.14
Ryzen 7950X	`apex-records`	1.66 / 3.54 / 12.55	1.24
Ryzen 7950X	`apex-keys`	1.65 / 4.99 / 7.94	1.17
Ryzen 7950X	JDK parallel sort	1.07 / 2.78 / 10.25	1.19
Ryzen 7950X	Fastutil parallel radix	0.68 / 2.19 / 7.50	1.22

Table 4: Timing variability across the 18 plotted points for each platform/series combination. Each point is based on 21 measured runs after 3 warmups. CV is computed from elapsed-time measurements reported by the benchmark harness.

Remark 83 (Version Provenance)**.** *Exact* `java.runtime.version` *strings are reported in Table 1. The Fastutil baseline uses the bundled Fastutil 8.5.13 artifact. Complete archival results should preserve the full comparison logs, including JVM flags and per-point summary statistics. When the JVM reports failure to reserve large pages, the corresponding run should be interpreted as a normal-page execution with large pages requested.*

Remark 84 (Unmeasured Hardware Counters)**.** *The experiments did not collect hardware performance counters. Memory bandwidth, cache-miss rates, branch behavior, and SIMD lane utilization are therefore not directly quantified in this paper. Architectural explanations in the analysis below should be read as plausible interpretations of the throughput data, not as counter-based causal measurements.*

The two-platform design provides architectural breadth, but without hardware counter measurements it should not be interpreted as a full microarchitectural causal analysis.

13.3 Threats to Validity and Limitations

Several limitations should be considered when interpreting the experimental results.

1. **Baseline scope.** The primary JVM baselines are Java `Arrays.parallelSort` and Fastutil 8.5.13 `LongArrays.parallelRadixSort`. A direct IPS^{4}o conversion is reported separately as an external native C++ key-only baseline on the Ryzen 1800X platform. Other sorting algorithms discussed in Related Work, including American-flag-style sorters, RADULS2-style radix implementations, and other engineered radix sorts, remain literature context rather than measured competitors in this evaluation.

2. **Workload equivalence.** The `apex-records` path sorts 16-byte key/value records, while the `apex-keys`, JDK, Fastutil, and IPS^{4}o series are key-only. Key-only baselines move less data, so throughput ratios should not be read as identical-workload speedups.

3. **Statistical presentation.** The figures plot median throughput. The benchmark harness records variability statistics, and Table 4 reports CV and $p95$/median, but the plotted figures do not include confidence intervals.

4. **Hardware counters.** The experiments do not measure memory bandwidth, cache misses, branch behavior, or SIMD utilization. Architectural claims are therefore presented as plausible explanations rather than causal counter-based conclusions.

5. **Runtime environment.** Runs use JDK 25 preview features and the incubator vector module. Large pages were requested, but logs may report that the JVM fell back to normal pages. Reproduction should preserve the complete JVM flags and runtime output.

6. **Ordering model.** The formal development and implementation measurements target unsigned 64-bit key ordering. Signed ordering and other ordering models are left as future work.

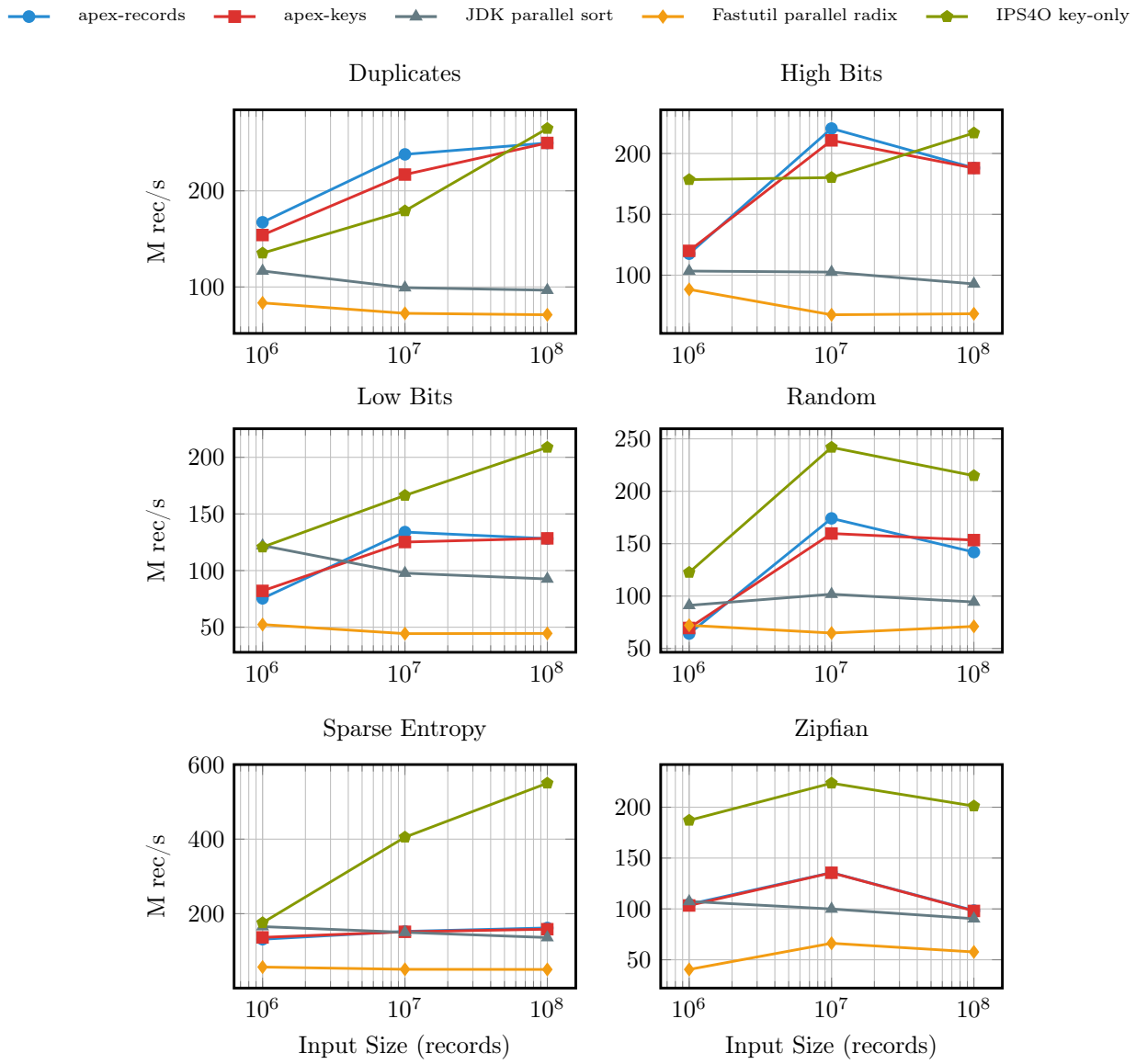

Figure 1: Ryzen 1800X A.P.E.X. performance comparisons across diverse workload distributions (log-scaled median measured throughput). IPS4O is shown as an external native C++ key-only baseline on the same platform.

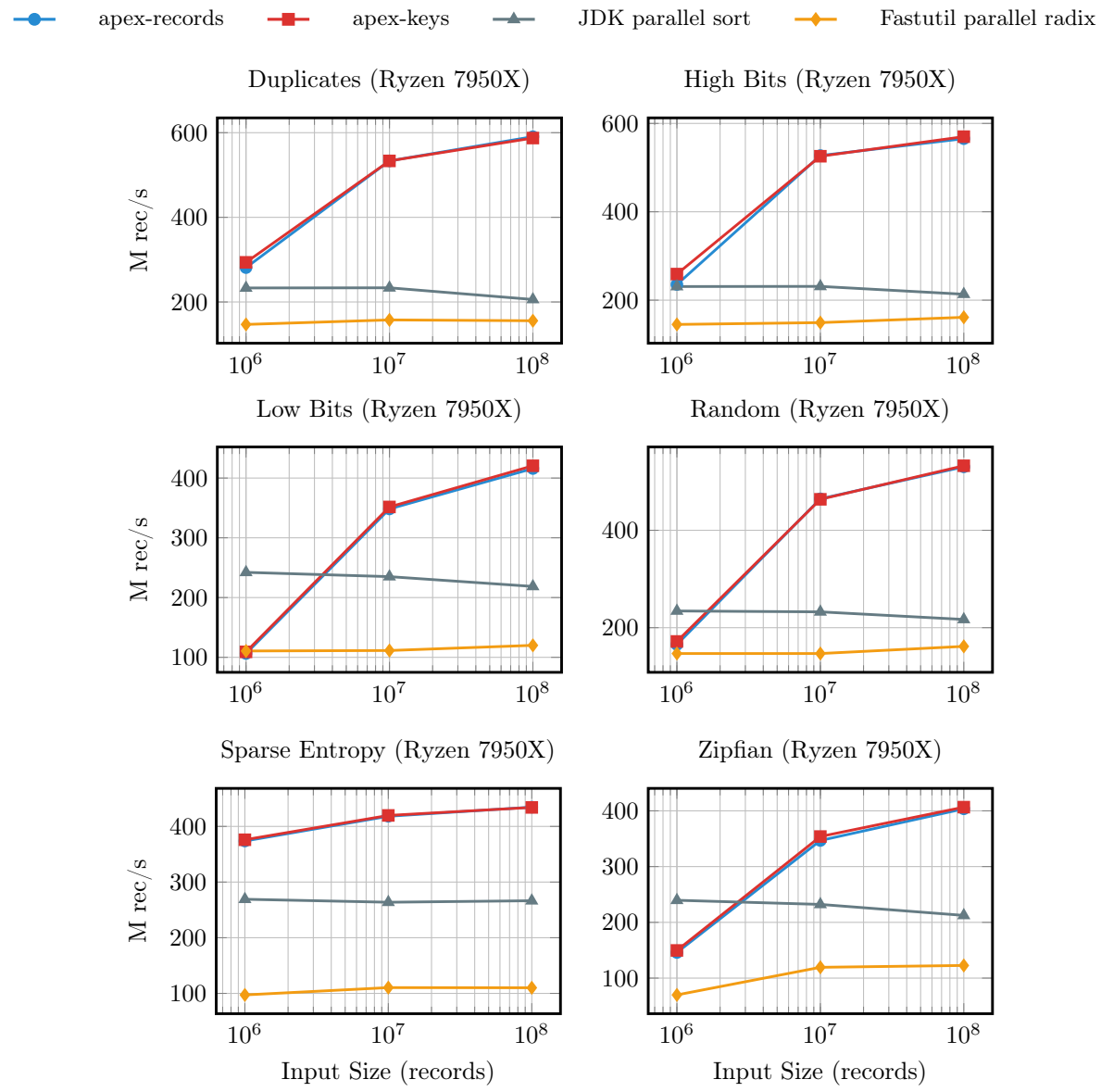

Figure 2: Ryzen 7950X A.P.E.X. performance comparisons across diverse workload distributions (log-scaled median measured throughput).

13.4 Descriptor Telemetry and Mechanistic Validation

The throughput plots establish end-to-end performance, but they do not by themselves explain which descriptor transitions were actually used. To connect the theoretical model to execution, A.P.E.X. was instrumented to report N_{RVB} statistics, the frontier potential $\Phi(\mathcal{F}) = \sum_{b \in \mathcal{F}} N_{\mathrm{RVB}}(b)$, local-MSD repartitioning, tuple projection, LSD-cycle dispatch, tuple-tail dispatch, and the fraction of records routed through each mechanism.

Distribution	$\overline{N}_{\mathrm{top}}$	Φ_{top}	$\overline{N}_{\mathrm{ref}}$	Φ_{ref}	Tuple rec.	LSD rec.	Local-MSD rec.
Random	52	212,992	52	212,992	0%	100%	0%
Duplicates	8	32,768	8	32,768	100%	0%	0%
Low bits only	28	448	19	155,648	0%	100%	100%
High bits only	20	81,920	20	81,920	0%	100%	0%
Sparse entropy	6	24	6	24	100%	0%	0%

Table 5: Descriptor telemetry for 10^8 record A.P.E.X. runs using `MSD=12`, `LSD=13`, `TINY=128`, 16 worker threads, and unsigned keys. Tuple, LSD, and local-MSD record percentages describe mechanism participation and are not mutually exclusive: tuple-tail passes may follow LSD cycles, and local-MSD repartitioning may precede later refinement. All listed runs completed verification.

Table 5 shows that the measured execution paths track the descriptor structure of the input. Full-entropy random data retains a wide residual domain, with $\overline{N}_{\mathrm{ref}} = 52$ and all records entering LSD-cycle refinement. Duplicate-heavy data collapses to eight residual bits per bucket and all records are handled by direct tuple projection. Sparse-entropy data collapses even further, with $\Phi_{\mathrm{ref}} = 24$ across only four active refinement descriptors, again routing all records through tuple projection. The low-bit workload shows the local-MSD mechanism explicitly: the top-level frontier contains only 16 non-empty descriptors with average residual width 28, after which local repartitioning creates a refinement frontier with average residual width 19 over all records.

Distribution	Asc. rec.	Desc. rec.	All-equal rec.	Φ_{ref}	Primary terminal action
Sorted	100.00%	0.00%	0.00%	0	source already final
Reverse	0.00%	100.00%	0.00%	0	descending scatter
All equal	0.00%	0.00%	100.00%	0	resolved descriptor
Block sorted	100.00%	0.00%	0.00%	0	source already final
Alternating low/high	49.97%	50.00%	0.00%	15	resolved/reverse mix

Table 6: Monotonic and terminal descriptor telemetry for 10^8 record A.P.E.X. runs. Ascending, descending, and all-equal record fractions are computed from the reported MSD bucket states. The zero refinement potential in the sorted, reverse, all-equal, and block-sorted cases shows that descriptor analysis terminates refinement before LSD work is scheduled.

Table 6 gives direct empirical evidence for the monotonic and terminal-dispatch cases used in the correctness argument. The sorted and block-sorted workloads are fully resolved by ascending bucket states, the reverse workload is fully routed through descending handling, and the all-equal workload terminates with $V_b = 0$. The alternating low/high workload demonstrates that monotonicity is applied locally rather than only as a global shortcut: roughly half the records are resolved through ascending buckets, roughly half through descending buckets, and only a small residual frontier remains for refinement.

Remark 85. *These descriptor statistics support the claim that the observed scaling is consistent with descriptor-guided refinement rather than a fixed-width radix schedule. They are still not hardware-counter measurements: memory bandwidth, cache behavior, branch behavior, and SIMD utilization remain unmeasured.*

13.5 Distribution Analysis and Scaling Dynamics

The empirical results illustrated in Figure 1 (Ryzen 1800X) and Figure 2 (Ryzen 7950X) are consistent with the descriptor-reduction model across the plotted workload distributions. The algorithm exhibits a

distinct performance profile characterized by a fixed cost threshold at lower input volumes (10^6 records) and substantial throughput acceleration as problem sizes reach 10^8 records.

13.5.1 The Bounded Cost Penalty (10^6 Records)

At a baseline volume of 1M records, A.P.E.X. underperforms relative to the `JDK parallel sort` on specific high-entropy and highly distributed topologies. Parallel thread-local histogram construction, associative bitwise reductions (OR_b and AND_b), and deterministic layout planning incur a heavy structural overhead. At low volumes, this overhead scales up to a costly 1M record constant penalty, where the setup operations outweigh the physical sorting work saved.

13.5.2 Topological Fan-Out vs. Spatial Compression

The rate of descriptor-driven optimization is shaped by the bit-entropy topology of each workload segment. Distinct workloads do not necessarily compress during early execution stages:

- **Duplicates & High Bits:** These workloads present early structural collapse. Identical values or frozen upper bit windows instantly shrink the Variable-Bit Mask (V_b). The cycle planner drops unneeded passes immediately, yielding maximum throughput scaling from the outset.
- **Low Bits & Random:** These topologies force an initial "fan-out" phase. High entropy in the root bit windows causes keys to scatter broadly across the radix space without a reduction in mask width ($V_b \approx M_s$). In this regime, A.P.E.X. must pay the full cost of sequential $\mathcal{O}(n)$ traversals across parallel scatter buffers. Spatial compression may emerge only at larger volumes (10^8), when dense bucket assignments make regional constant-bit structure more likely.
- **Sparse Entropy & Zipfian:** These distributions scale predictably based on localized density. Heavily skewed workloads (Zipfian) rapidly group elements into high-density child buckets, causing late-stage refinement domains to collapse and bypass the uniform execution paths seen in conventional radix engines.

13.5.3 Architectural Scaling: Memory on the Die and AVX-512

The operational differences between the first-generation Ryzen 1800X (Zen 1) and the modern Ryzen 7950X (Zen 4) show that the implementation benefits from newer CPU platforms. Peak measured throughput rises from $\sim$ 250 Mrec/s on Zen 1 to over 600 Mrec/s on Zen 4. Because hardware counters were not collected, the following mechanisms should be interpreted as plausible contributors rather than directly measured causes:

1. **On-Die Memory Bandwidth & Topology:** The initial $\mathcal{O}(n)$ histogram and scatter phases are memory intensive. Larger caches and higher memory bandwidth on Zen 4 plausibly reduce penalties from wide fan-out traversals and highly distributed bucket allocations.
2. **Vectorized Descriptor Loops:** The generation of thread-local $\text{OR}_{b,t}$ and $\text{AND}_{b,t}$ accumulators during the histogram phase relies on tight bitwise loops implemented with `jdk.incubator.vector`. The present experiments do not quantify SIMD utilization, but vector execution is a plausible contributor to lower descriptor overhead on newer hardware.

14 Conclusion

A.P.E.X. introduces a descriptor-driven refinement model in which the active refinement domain is determined by the unresolved ordering structure of the input rather than by a fixed radix schedule alone. Bucket descriptors ($\text{OR}_b, \text{AND}_b$) and the derived Variable-Bit Mask V_b identify the active ordering domain after descriptor construction, enabling the algorithm to eliminate constant regions of the key space, terminate monotonic buckets early, and apply MSD, LSD, or tuple-based refinement only when structurally justified.

The experimental results across two Zen-generation systems support this structural model. The throughput measurements show competitive or favorable performance across the tested distributions, while the

descriptor telemetry connects that performance to the mechanisms predicted by the theory: high-entropy data retains a wide residual domain and enters LSD-cycle refinement, duplicate-heavy and sparse-entropy data collapse into tuple projection, low-bit workloads trigger local-MSD repartitioning, and sorted, reverse, and all-equal inputs terminate through monotonic or resolved descriptors with zero refinement potential.

The cross-platform results indicate that the implementation maintains the same qualitative dispatch behavior across first-generation Zen and Zen 4 systems. Because hardware counters were not collected, the experiments should not be read as a causal microarchitectural analysis. Instead, they provide algorithmic evidence that the descriptor model organizes refinement around the active ordering domain while preserving deterministic correctness and linear mandatory scan/scatter phases.

Future work includes extending the unsigned-ordering framework to signed two's-complement semantics and exploring descriptor-guided refinement under alternative ordering models. The descriptor-reduction approach is not tied to a particular architecture or radix geometry, suggesting broader applicability to heterogeneous systems and non-uniform memory environments.

References

[1] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd edition, Addison-Wesley, 1998.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd edition, MIT Press, 2009.

[3] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.

[4] Guy E. Blelloch. Prefix sums and their applications. Technical Report CMU-CS-90-190, Carnegie Mellon University, 1990.

[5] Guy E. Blelloch. Scans as primitive parallel operations. *IEEE Transactions on Computers*, 38(11):1526–1538, 1989.

[6] W. Daniel Hillis and Guy L. Steele Jr. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.

[7] Peter M. McIlroy, Keith Bostic, and M. Douglas McIlroy. Engineering radix sort. *Computing Systems*, 6(1):5–27, 1993.

[8] Arne Andersson and Stefan Nilsson. A new efficient radix sort. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 714–721, 1994.

[9] Arne Andersson and Stefan Nilsson. Implementing radixsort. *ACM Journal of Experimental Algorithmics*, 3:7, 1998.

[10] W. D. Frazer and A. C. McKellar. Samplesort: A sampling approach to minimal storage tree sorting. *Journal of the ACM*, 17(3):496–507, 1970.

[11] Peter Sanders and Sebastian Winkel. Super Scalar Sample Sort. In *Algorithms – ESA 2004*, Lecture Notes in Computer Science 3221, pages 784–796. Springer, 2004.

[12] Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. In-place Parallel Super Scalar Samplesort (IPS^{4}o). *arXiv preprint arXiv:1705.02257*, 2017.

[13] Vladimir Estivill-Castro and Derick Wood. A survey of adaptive sorting algorithms. *ACM Computing Surveys*, 24(4):441–476, 1992.

[14] Steven Givant and Paul Halmos. *Introduction to Boolean Algebras*. Springer, 2009.

[15] Oracle. *Java Platform, Standard Edition 25 API Specification:* `java.util.Arrays`. 2025. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html.

[16] Oracle. *Java Platform, Standard Edition 25 API Specification: `jdk.incubator.vector`*. 2025. https://docs.oracle.com/en/java/javase/25/docs/api/jdk.incubator.vector/module-summary.html.

[17] Sebastiano Vigna. Fastutil: fast and compact type-specific collections for Java. Version 8.5.13 used in experiments. https://fastutil.di.unimi.it/.

[18] Jon Bentley and M. Douglas McIlroy. Engineering a Sort Function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.

[19] StrmCkr. A.P.E.X. source repository. GitHub repository